
Advanced Strategies for Remote Client & Distributed AI Inference

Web • Android • iOS — A Technical Reference for Infrastructure Architects

Author: Perplexity Computer | March 2026

1. Executive Summary

Three years ago, running a language model on a phone was a toy demo. Today, billion-parameter models execute in real time on flagship devices.¹ This shift came not from faster chips alone, but from rethinking how we build, compress, and deploy models. The convergence of efficient architectures (deep-thin, MoE, hybrid attention), aggressive quantization (4-bit and below), and mature runtimes (ExecuTorch 1.0, llama.cpp, MLX, MLC-LLM) has made on-device inference practical across every major platform.

In the browser, WebGPU-backed frameworks like WebLLM retain up to 80% of native GPU performance,³ while WebNN stabilizes access to NPUs. On Android, ExecuTorch (GA October 2025)²⁵ with XNNPACK/Vulkan/Qualcomm backends and Arm SME2 delivering 3.9x speedups⁹ define the production stack. On iOS, the combination of CoreML (CPU/GPU/ANE dispatch), MLX with unified memory,¹¹ and Metal-backed llama.cpp enables 7B Q4 inference at ~9 tok/s on M-series chips.¹²

For models too large for a single device, distributed frameworks offer real solutions: Exo clusters heterogeneous devices with RDMA over Thunderbolt achieving 99% latency reduction,⁷ Petals runs Llama 2 70B at 6 tok/s BitTorrent-style,⁸ and llama.cpp's RPC backend enables multi-machine GPU inference.¹⁸ Hybrid edge-cloud strategies — speculative decoding (FlexSpec, DSSD)¹⁴¹⁵ and split computing²⁸ — dynamically balance latency, accuracy, and bandwidth.

Key Takeaways: (1) 3-4B Q4 models are the sweet spot for on-device inference across platforms. (2) WebGPU is the workhorse for browser LLMs; WASM is the universal fallback. (3) ExecuTorch is the convergence framework for mobile (Android + iOS). (4) Distributed inference is production-ready for self-hosted clusters. (5) Speculative decoding is the highest-leverage hybrid optimization.

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

3. WebLLM: High-Performance In-Browser LLM Inference Engine, <https://arxiv.org/abs/2412.15803>

7. Exo: Run frontier AI locally, <https://github.com/exo-explore/exo>

8. Petals: Run LLMs at home, BitTorrent-style, <https://petals.dev>

9. Accelerating on-device ML with ExecuTorch and Arm SME2, <https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/accelerating-on-device-ml-inference-with-executorch-and-arm-sme2>

11. WWDC 2025 — Explore LLM on Apple silicon with MLX, <https://dev.to/arshtechpro/wwdc-2025-explore-llm-on-apple-silicon-with-mlx-1if7>

12. llama.cpp Performance on Apple Silicon A-series, <https://github.com/ggml-org/llama.cpp/discussions/4508>

14. FlexSpec: Frozen Drafts Meet Evolving Targets in Edge-Cloud SD, <https://arxiv.org/abs/2601.00644>

15. DSSD: Distributed Split Speculative Decoding, <https://icml.cc/virtual/2025/47046>

18. llama.cpp RPC backend, <https://github.com/ggml-org/llama.cpp/blob/master/tools/rpc/README.md>
25. ExecuTorch: On-device ML on Meta's family of apps, <https://engineering.fb.com/2025/07/28/android/executorch-on-device-ml-meta-family-of-apps/>
28. Split Computing: DNN Inference Partition — ScienceDirect, <https://www.sciencedirect.com/science/article/pii/S2665917422000435>

2. On-Device Model Landscape (2026)

The on-device model ecosystem has matured dramatically. The table below captures the best models by size category, from ultra-small 125M classifiers to 8B instruction-following models.¹

Model	Params	Key Strength	Notes
MobileLLM (Meta)	125M–350M	Deep-thin arch, 50 tok/s on iPhone	MobileLLM-R1: 2-5x reasoning vs 2x larger models
SmolLM2 (HuggingFace)	135M, 360M, 1.7B	11T training tokens	Outperforms Llama 3.2 1B at 1.7B
Gemma 3 (Google)	270M–27B	Extreme efficiency at small sizes	270M viable for classification/embedding
Qwen2.5/3 (Alibaba)	0.5B–72B	Strong multilingual, reasoning	Qwen3-4B rivals Qwen2.5-72B on reasoning
Phi-4 (Microsoft)	3.8B (mini), 14B	Phi-4-reasoning rivals o1-mini	Math/code specialist
Llama 3.2 (Meta)	1B, 3B	128K ctx, Qualcomm/MediaTek optimized	Best ecosystem support
Liquid AI LFM 2.5	0.5B–3.2B	Linear-time inference, no KV cache growth	State-space architecture
BitNet (Microsoft)	2B	Native 1.58-bit, fits in 400MB	CPU-efficient, no GPU required

Architecture Trends

- Deep-thin architectures: More layers with smaller hidden dimensions consistently outperform wide-shallow designs below 1B parameters. MobileLLM proved this counterintuitive finding — architecture matters more than parameter count at small scale.¹
- Mixture of Experts (MoE): Over 60% of frontier model releases since early 2025 adopt MoE. DeepSeek-V3 uses 256 experts with fine-grained routing. Qwen3-30B-A3B activates only 3B parameters yet outperforms QwQ-32B.¹
- Hybrid attention: Combining Mamba state-space models with attention (Qwen3 Next, Nvidia-Nemotron3). Linear attention and LIV convolutions reduce decode complexity from $O(n^2)$ to $O(n)$.
- BitNet 1.58-bit: Native ternary training (-1, 0, 1) eliminates floating-point multiply. A 2B BitNet model fits in 400MB and runs efficiently on CPU alone.¹
- Native multi-modal: Qwen3 Omni and Gemini 3 handle text, vision, and audio in a single model; SmolVLM-256M achieves vision understanding under 1GB.

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

3. Quantization Strategies for Edge

Quantization is the single most impactful optimization for edge deployment. The transition from FP16 to 4-bit reduces memory by 4x and proportionally reduces memory bandwidth pressure — the critical bottleneck on mobile

devices with only 50-90 GB/s.¹

Method	Bits	Memory Reduction	Quality Impact	Best For
FP16 (baseline)	16	1x (reference)	None	Server, no constraints
GPTQ	4	4x	1-3% drop	Post-training, GPU inference
AWQ	4	4x	1-2% drop	19M+ HF downloads, GPU/NPU
SpinQuant	4 (W+ A+KV)	<3% loss	Weights + activations + KV	Mobile production
ParetoQ	2-3	5-8x	Model learns new repr.	Extreme edge (2-bit > smaller 4-bit)
BitNet	1.58	~10x	Native training quality	CPU-only, no GPU needed
SlimLLM (mixed)	2-4 mixed	4-6x	Layer-adaptive	Heterogeneous precision per layer
mxfp4	4 (microscaling)	4x	Hardware-native	Apple A19 Pro, future NPUs
KV Cache Quant	3-4 bits	4-5x KV savings	Negligible at 3-bit	Long context on device

Practical Recommendations

- Default choice: Q4_K_M (GGUF) for general-purpose deployment. Superior accuracy vs naive Q4_0 with minimal overhead.²
- KV cache: Quantize to Q8 minimum; 3-bit KV is viable with careful validation. KV cache grows linearly with sequence — at 4K context for an 8B model, expect ~1 GB.¹
- Sub-4-bit: ParetoQ shows that a larger model at 2-bit outperforms a smaller model at 4-bit. Consider aggressive quantization of big models over deploying smaller full-precision ones.
- Mixed precision (SlimLLM): Different layers tolerate different quantization levels. Attention projection layers are more sensitive; FFN layers tolerate aggressive compression.
- Hardware-native formats: Apple A19 Pro supports mxfp4 (microscaling FP4) natively at ~35 TOPS. Target hardware-native quant formats when available.¹
- GGUF is the de facto standard: Streaming-friendly, includes metadata and tensor layout, versioned, with broad model availability on HuggingFace.²³

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

2. Ship Private, Offline AI in 2025 — debugg.ai, <https://debugg.ai/resources/ship-private-offline-ai-2025-on-device-llm-webgpu-webnn-wasm>

23. llama.cpp quantize README, <https://github.com/ggml-org/llama.cpp/blob/master/tools/quantize/README.md>

4. Web/Browser Inference

Browser-based LLM inference is now a solved engineering problem. WebGPU provides near-native GPU compute; WebNN maps to platform ML accelerators; WASM ensures universal fallback. The combination delivers private, offline, zero-server-cost inference to billions of browser users.²

Execution Backends

Backend	Compute Model	Ideal For	Status (2026)
WebGPU	Compute shaders, storage buffers, subgroup ops	Matmul-heavy decode, KV cache, attention kernels	Stable: Chrome/Edge desktop+Android, Safari macOS/iOS 17+, Firefox shipping
WebNN	High-level API → DirectML / CoreML / NNAPI	ONNX graphs on NPU/DSP; mobile SoCs	Chrome behind flag/origin trial; stabilizing across Chromium
WASM (SIMD+ threads)	CPU vector ops, SharedArrayBuffer	Fallback; tokenization; control logic; small models	Ubiquitous (requires cross-origin isolation for threads)

Capability Detection & Progressive Enhancement

Feature-detect at runtime: probe `navigator.gpu`, `navigator.ml`, `hardwareConcurrency`, `deviceMemory`. Use `GPUAdapter.limits.maxStorageBufferBindingSize` to estimate workable tensor sizes. Build a runtime plan: backend selection → model size → quantization → max context → thread count.²

Device Tier	Backend	Model	Quant	Max Ctx	KV Quant
Desktop dGPU (≥512MB buffer)	WebGPU	Llama 3.1 8B	q4_k_m	4096	Q8
Mid-tier iGPU / mobile high-end	WebGPU	Phi-3 mini 3.8B	q4_0	3072	Q8
WebNN + ≥4GB RAM	WebNN	Phi-3 mini 3.8B	q4_0	3072	Q8
≥8GB RAM (no GPU/WebNN)	WASM	Phi-3 mini 3.8B	q4_0	2048	Q8
Fallback	Server	Tiny 1B	q8_0	1024	—

Key Frameworks

Framework	Backend	Format	Strengths
WebLLM (MLC)	WebGPU (compiled WGSL kernels via TVM)	MLC shards	Up to 80-85% native perf; OpenAI-compatible API; shard-based loading for mobile
Transformers.js	WebGPU + CPU fallback	ONNX (large single files)	Latest models fast; pipelines for non-LLM tasks; desktop-first
ONNX Runtime Web	WebGPU EP + WebNN EP	ONNX INT4/INT8	Broad operator coverage; safety classifiers; WebNN for NPU
llama.cpp-WASM	WASM + WebGPU	GGUF	Battle-tested; streaming decode; KV cache quant; wide quant support

Memory Budgeting

Total memory = Weights + KV cache + Activations + Fragmentation overhead. For an 8B model at Q4_K_M: weights ≈ 4.5-5.5 GB. KV cache at Q8 with 4K context: $2 \times 4096 \times 4096 \times 32 \times 1 \text{ byte} \approx 1.07 \text{ GB}$. Budget 20-30% headroom for prefill activations.² Mobile: cap context at 1-2K tokens and prefer 3-4B models.

Loading Architecture: Streaming Shards + Service Worker Caching

- Serve model in 64-128 MB shard files over HTTP with Range request support
- Cache Storage API for byte-level caches with versioned URLs; IndexedDB for tensor metadata and tokenizers²
- Service Worker intercepts requests, enables offline revalidation, supports resume after interruption

- MLC/WebLLM splits models into shards (similar to gguf-split) — better for mobile than large single ONNX files

Browser Coverage Matrix

Browser	WebGPU	WebNN	WASM SIMD+Threads	Notes
Chrome/Edge (Desktop)	Stable	Behind flag/OT	Full	Best WebGPU support; ORT WebGPU EP works well
Chrome (Android)	Stable	In development	Full	Mobile WebGPU viable on high-end
Safari (macOS Sonoma+)	Supported	N/A (CoreML native)	Full	Test memory limits carefully
Safari (iOS 17/18)	Supported (limits)	N/A	Full	Tighter memory; test thoroughly
Firefox	Shipping	N/A	Full	Performance varies across platforms

Production Blueprint

1. Capability plan: detect backends + adapter limits → choose model/quant/ctx. 2. Storage: Service Worker caches shards with hash verification; IndexedDB for metadata. 3. Init: spin up Worker, compile pipelines, allocate reusable buffers. 4. Generate: stream tokens with safety redaction (PII regex, TinyBERT classifier via ORT). 5. Fallback: downshift model/ctx on OOM; offer server inference if online.²

2. Ship Private, Offline AI in 2025 — debugg.ai, <https://debugg.ai/resources/ship-private-offline-ai-2025-on-device-llm-webgpu-webnn-wasm>
3. WebLLM: High-Performance In-Browser LLM Inference Engine, <https://arxiv.org/abs/2412.15803>
4. AI In Browser With WebGPU: 2025 Developer Guide, <https://aicompetence.org/ai-in-browser-with-webgpu/>
22. WebLLM Blog — MLC-LLM, <https://blog.mlc.ai/2024/06/13/webllm-a-high-performance-in-browser-llm-inference-engine>
26. Transformers.js vs WebLLM — Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1lw6jz5/transformersjs_vs_webllm/

5. Android Inference

Android on-device inference has consolidated around ExecuTorch as the primary production framework, with llama.cpp and MLC-LLM as proven alternatives. Meta deploys ExecuTorch across Instagram, WhatsApp, Messenger, and Facebook, processing billions of daily inferences.²⁵

Runtime Stack

Framework	Backends	Format	Production Status
ExecuTorch 1.0 (Meta)	XNNPACK (CPU), Vulkan (GPU), Qualcomm QNN, MediaTek NeuroPilot	.pte (exported PyTorch)	GA Oct 2025; 50KB base; 12+ HW backends; 80%+ HF edge LLMs work OOB
MLC-LLM	OpenCL (Adreno/Mali GPUs), Vulkan	MLC compiled	Cross-platform; OpenAI-compatible API; strong on Qualcomm/Mali
llama.cpp (NDK)	CPU (NEON/SVE2), Vulkan, OpenCL	GGUF	NDK compilation; wide quant support; community ecosystem
TFLite (legacy)	GPU delegate, Qualcomm SNPE (direct)	TFLite flatbuf	NNAPI deprecated; migrate to GPU delegate or direct vendor SDK

Hardware Performance Characteristics

- Memory bandwidth: 50-90 GB/s on flagship SoCs (vs 2-3 TB/s datacenter). This 30-50x gap makes decode fundamentally memory-bound.¹
- big.LITTLE threading: Schedule compute on performance cores (Cortex-X series) during decode; use efficiency cores for background tasks. Thread count should match performance core count.
- NEON SMMLA/SDOT instructions: ARM v8.6+ matrix multiply and dot-product instructions deliver up to 4x speedup for quantized matmul on CPU.¹
- Arm SME2: Scalable Matrix Extension 2 achieves 3.9x speedup for FP16 workloads (demonstrated with SqueezeSAM on ExecuTorch for Instagram's cutout tool).⁹
- NPU TOPS: Qualcomm Snapdragon 8 Elite Gen 5 ~60 TOPS; MediaTek Dimensity 9400+ ~50 TOPS. Both optimized for Llama 3.2.

ExecuTorch Deep Dive

ExecuTorch 1.0 (GA October 2025) is PyTorch's inference framework for edge.²⁵ Key characteristics:

- 50KB base runtime footprint — suitable from microcontrollers to smartphones
- PyTorch 2.x export: `torch.export` → .pte file with operator fusion and backend delegation
- Backend delegation: XNNPACK (optimized CPU kernels), Vulkan (cross-vendor GPU), Qualcomm QNN (Hexagon DSP + NPU), MediaTek NeuroPilot
- Over 80% of popular edge LLMs on HuggingFace work out-of-the-box with ExecuTorch
- NNAPI is deprecated — migrate to GPU delegate or direct vendor SDKs (Qualcomm SNPE, Arm NN)²⁷

Reference Implementation: PocketPal AI

PocketPal AI¹⁹ is an open-source React Native app demonstrating production on-device inference: GGUF model loading from HuggingFace, offline chat, local-only processing. Available on both Android and iOS. Uses llama.cpp as the inference backend with native bindings.

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

9. Accelerating on-device ML with ExecuTorch and Arm SME2, <https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/accelerating-on-device-ml-inference-with-executorch-and-arm-sme2>

19. PocketPal AI, <https://github.com/a-ghorbani/pocketpal-ai>

25. ExecuTorch: On-device ML on Meta's family of apps, <https://engineering.fb.com/2025/07/28/android/executorch-on-device-ml-meta-family-of-apps/>

27. NNAPI Migration Guide — Android Developers, <https://developer.android.com/ndk/guides/neuralnetworks/migration-guide>

6. iOS/iPhone Inference

Apple's ecosystem offers three primary paths for on-device inference: ExecuTorch with CoreML backend, MLX (Apple's open-source framework), and llama.cpp with Metal. The unified memory architecture of Apple Silicon is a fundamental advantage — CPU, GPU, and ANE share the same memory pool, eliminating costly data transfers.¹¹

Runtime Stack

Framework	Backend	Key Features	Best For
ExecuTorch + CoreML	CPU / GPU / Apple Neural Engine	Dynamic dispatch; FP32/FP16; ≥iOS 16; PyTorch 2.x export	Production apps requiring ANE acceleration

Framework	Backend	Key Features	Best For
MLX (Apple, WWDC 2025)	Metal GPU, unified memory	Swift API; lazy eval; on-device LoRA fine-tuning; mlx-lm ecosystem	Apple Silicon native; research-to-production
llama.cpp + Metal	Metal GPU shaders	GGUF format; wide quant support; community benchmarks	Maximum flexibility; broad model support
CoreML (standalone)	CPU / GPU / ANE	coremltools conversion from PyTorch/TF; .mlmodel/.mlpackage	Vision/audio models; Apple-optimized pipelines

MLX: Apple's ML Framework (WWDC 2025)

MLX is Apple's open-source machine learning framework designed specifically for Apple Silicon.¹¹ Key differentiators:

- Unified memory: CPU and GPU share memory — no data copying between devices. Critical for large model inference.
- Metal GPU acceleration: Direct Metal integration for matrix operations, attention kernels.
- Swift API: Native Swift bindings for iOS/macOS integration alongside Python API.
- On-device fine-tuning: LoRA and QLoRA fine-tuning directly on device — personalize models without cloud.
- Lazy evaluation: Operations only compute when results are needed, reducing peak memory.

Performance Benchmarks

Device	Model	Quant	Prefill (tok/s)	Decode (tok/s)	Backend
iPhone (A-series)	1B Q4_0	4-bit	~400	~39-55	Metal
iPhone (A-series)	MobileLLM 125M	—	—	~50	CPU
M2 Max MacBook	7B Q4	4-bit	—	~40	Metal (38 GPU cores)
M2 Max MacBook	13B Q4	4-bit	—	~24	Metal
M-series (general)	7B Q4	4-bit	—	~9	Metal
Apple A19 Pro (NPU)	—	mxfp4	~35 TOPS	—	Neural Engine

CoreML Backend Details

CoreML¹³ provides dynamic dispatch across CPU, GPU, and Apple Neural Engine (ANE). The runtime automatically selects the optimal compute unit based on model operations, device thermal state, and power budget. Supports FP32 and FP16 computation. Minimum requirements: iOS 16+, macOS 13+.

- ANE excels at quantized operations (INT8/INT4) and standard attention patterns
- GPU (Metal) handles custom operations and large batch sizes
- CPU handles operations not supported by GPU/ANE
- Apple A19 Pro: ~35 TOPS with mxfp4 hardware support¹

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

10. ExecuTorch on iOS — PyTorch documentation, <https://docs.pytorch.org/executorch/stable/using-executorch-ios.html>

11. WWDC 2025 — Explore LLM on Apple silicon with MLX, <https://dev.to/arshtechpro/wwdc-2025-explore-llm-on-apple-silicon-with-mlx-1if7>

12. llama.cpp Performance on Apple Silicon A-series, <https://github.com/ggml-org/llama.cpp/discussions/4508>

13. Core ML Backend — ExecuTorch documentation, <https://docs.pytorch.org/executorch/1.0/ios-coreml.html>

7. Distributed Inference Frameworks

When a model exceeds the capacity of a single device, distributed inference splits the workload across multiple machines or devices. The field has matured from research prototypes to production-ready frameworks with distinct architectural approaches.

Framework	Architecture	Key Performance	Use Case
Petals	BitTorrent-style P2P; volunteer GPU network	6 tok/s Llama 2 70B; 4 tok/s Falcon 180B	Community inference; fine-tuning; Llama 3.1 405B support
Exo	Heterogeneous P2P clustering; no master-worker	99% latency reduction (RDMA/TB); 3.2x speedup on 4 devices	Self-hosted clusters; mixed device types (Mac/Linux/iOS/Android)
llama.cpp RPC	Client-server RPC; distributed GPU layers	Multi-machine GPU inference; FP16 + quantized models	Jetson clusters; multi-GPU across network; llama.cpp ecosystem
MDI-LLM	Recurrent pipeline parallelism; layer partitioning	Reduced idle time; parallel multi-sequence generation	Low-power edge networks; collaborative edge devices
EdgeShard	Dynamic programming partitioning; edge-cloud hybrid	50% latency reduction; 2x throughput vs baselines	Heterogeneous edge + cloud; adaptive device selection
vLLM	Head/worker mode; PagedAttention	Production-grade serving; continuous batching	Multi-machine datacenter; high-throughput serving

Exo: Deep Dive

Exo⁷ connects all your devices into a unified AI cluster. Unlike master-worker architectures, Exo uses true P2P device equality — any connected device can participate. Key innovations:

- Automatic device discovery: Zero-configuration network discovery; devices find each other automatically
- RDMA over Thunderbolt 5: Day-0 support achieves 99% latency reduction between devices vs TCP networking
- Topology-aware auto-parallel: Real-time assessment of device topology, resources, and inter-device bandwidth to determine optimal model splitting
- Tensor parallelism: Model sharding across devices — 1.8x speedup on 2 devices, 3.2x on 4 devices
- Ring memory-weighted partitioning: Each device runs layers proportional to its available memory
- Multi-backend: MLX and tinygrad inference engines; interoperable

llama.cpp RPC Backend

The RPC backend in llama.cpp¹⁸ enables distributing GPU inference across multiple machines on the same network. Each remote machine runs an `rpc-server` that exposes its accelerators. The client connects to multiple RPC servers and distributes model layers across them.²⁴

- Build with `-DGGML_RPC=ON -DGGML_CUDA=ON` for CUDA+RPC support
- Supports heterogeneous hardware: mix CUDA GPUs, CPUs, and different accelerators
- Proven on Jetson clusters for edge distributed inference
- Quantized model support (after initial FP16-only limitation was resolved)

Petals

Petals⁸ enables running models up to 405B parameters by splitting them across volunteer GPU nodes. Each user loads a subset of layers; the network collaboratively serves the full model. Supports Llama 3.1 405B, Mixtral 8x22B, Falcon 180B, BLOOM 176B. Fine-tuning and inference up to 10x faster than offloading.

-
- 6. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing, <https://arxiv.org/abs/2405.14371>
 - 7. Exo: Run frontier AI locally, <https://github.com/exo-explore/exo>
 - 8. Petals: Run LLMs at home, BitTorrent-style, <https://petals.dev>
 - 17. MDI-LLM: Model-Distributed Inference for LLMs at the Edge, <https://arxiv.org/html/2505.18164v1>
 - 18. llama.cpp RPC backend, <https://github.com/ggml-org/llama.cpp/blob/master/tools/rpc/README.md>
 - 24. Distributed llama.cpp on Jetson (RPC Mode), https://wiki.seeedstudio.com/ai_robotics_distributed_llama_cpp_rpc_jetson/

8. Hybrid Edge-Cloud Strategies

Pure on-device and pure cloud are extremes. Hybrid strategies dynamically partition workloads between edge devices and cloud/edge servers, optimizing for latency, accuracy, bandwidth, and cost. This section covers the most impactful hybrid patterns.

Split Computing

Split computing partitions a DNN model at a dynamic layer boundary: early layers run on the device, later layers on the edge/cloud server. The splitting point adapts based on wireless channel conditions, device compute capacity, and model sensitivity.²⁸

- Dynamic splitting: reinforcement learning selects optimal split point per inference
- Reduces device compute footprint by 60%+ while containing end-to-end latency
- Activation compression between split point reduces uplink bandwidth

Speculative Decoding (Edge-Cloud)

The highest-leverage hybrid pattern: run a small draft model on the edge device, verify tokens with a large cloud model. Correct tokens are accepted without cloud latency; rejected tokens are regenerated. Up to 2-3x speedup with exact quality preservation.¹

FlexSpec: Shared-Backbone Architecture

FlexSpec¹⁴ addresses the tight coupling problem in edge-cloud speculative decoding. Key innovation: a shared-backbone architecture where a single static edge-side draft model remains compatible with a large family of evolving cloud-side target models. This eliminates repeated model synchronization overhead, enabling scalable speculative decoding in dynamic edge-cloud environments.

- Frozen draft model: no need to retrain/update when cloud model evolves
- Channel-aware adaptive speculation: adjusts speculation depth based on network conditions
- Communication-efficient: reduces synchronization overhead vs standard SD

DSSD: Distributed Split Speculative Decoding

DSSD¹⁵ introduces a three-tier architecture: SLM on device → LLM at edge → split verification between device and edge. By partitioning the verification phase, DSSD replaces expensive uplink transmission of candidate tokens with local partial verification, reducing communication overhead while preserving accuracy.

EdgeSD: Speculative Decoding for Multimodal LLMs

EdgeSD²¹ extends speculative decoding to multimodal LLMs (vision-language models) at the edge, enabling efficient inference for image understanding and visual QA tasks on resource-constrained devices.

Federated Learning: NVIDIA FLARE + ExecuTorch

NVIDIA and Meta's integration of NVIDIA FLARE with ExecuTorch¹⁶ brings federated learning to mobile devices at scale. Key capabilities:

- Hierarchical FL: millions of devices contribute to model training without sharing raw data
- On-device training via ExecuTorch; aggregation via FLARE's secure aggregation protocols
- Privacy-preserving: data never leaves the device; only model updates are communicated
- Domain-agnostic: applies to NLP, vision, recommendation systems

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

14. FlexSpec: Frozen Drafts Meet Evolving Targets in Edge-Cloud SD, <https://arxiv.org/abs/2601.00644>

15. DSSD: Distributed Split Speculative Decoding, <https://icml.cc/virtual/2025/47046>

16. Federated Learning on Mobile with NVIDIA FLARE and ExecuTorch,

<https://developer.nvidia.com/blog/effortless-federated-learning-on-mobile-with-nvidia-flare-and-meta-executorch/>

21. EdgeSD: Speculative Decoding with Vision LLMs, <https://ieeexplore.ieee.org/document/11409366/>

28. Split Computing: DNN Inference Partition — ScienceDirect, <https://www.sciencedirect.com/science/article/pii/S2665917422000435>

9. Memory & Bandwidth Optimization

The fundamental constraint of on-device LLM inference is memory bandwidth, not compute. Mobile devices offer 50-90 GB/s vs datacenter GPUs at 2-3 TB/s — a 30-50x gap.¹ Autoregressive decode is memory-bound: the entire model weights are loaded for each generated token. Every optimization in this section targets reducing memory traffic or bounding memory growth.

The Bandwidth Gap

Platform	Memory Bandwidth	Available RAM	Typical Bottleneck
NVIDIA H100 (datacenter)	3.35 TB/s (HBM3)	80 GB	Compute-bound at batch > 1
NVIDIA A100	2.0 TB/s (HBM2e)	40/80 GB	Compute-bound at batch > 1
Apple M4 Max	~546 GB/s (unified)	36-128 GB	Bandwidth-bound at large batch
Qualcomm SD 8 Elite	~77 GB/s (LPDDR5X)	12-24 GB	Always bandwidth-bound
MediaTek Dim. 9400+	~68 GB/s (LPDDR5X)	8-16 GB	Always bandwidth-bound
Typical Android flagship	50-90 GB/s	8-16 GB	Always bandwidth-bound
Low-end mobile	~30 GB/s	4-6 GB	Both bandwidth and memory limited

KV Cache Management

The KV cache grows linearly with sequence length and can exceed model weights for long contexts. For an 8B model at FP16 with 4K context: ~2 GB of KV cache alone. Management strategies:¹

Technique	Mechanism	Result
StreamingLLM	Fixed attention window + initial tokens	Infinite-length generation with fixed memory
ChunkKV	Chunk-level instead of token-level eviction	26% throughput improvement over token-level
EvoKV	Evolutionary selection of critical KV entries	Matches full KV quality with 1.5% budget
KV Quantization (3-bit)	Compress KV tensors to 3 bits	4-5x KV memory reduction; negligible quality drop

Technique	Mechanism	Result
Sliding Window Attention	Bound attention to fixed window size	Constant KV size; loses long-range dependencies

MoE Memory Optimization: EdgeMoE

Mixture-of-Experts models load all experts into memory despite sparse activation. EdgeMoE partitions inactive experts to external storage (flash/SSD), loading them on-demand: 5-18% memory reduction with 1.2-2.7x inference speedup through predictive expert prefetching.¹

FlashAttention for Mobile

- FlashAttention-2: achieves 72% model FLOPs on A100s — the standard for efficient attention¹
- FlashAttention-3: 75% utilization (740 TFLOPs/s) on H100s
- FlashAttention-4: 20% speedup on Blackwell architecture
- Mobile variants: adapted for Metal (Apple) and Vulkan (Android) with reduced memory footprint
- Multi-token prediction: generates multiple tokens per forward pass at near-zero latency cost

1. On-Device LLMs: State of the Union, 2026, <https://v-chandra.github.io/on-device-llms/>

10. Reference Architecture & Decision Matrix

Decision Tree: When to Use Which Approach

Select your inference strategy based on model size, latency requirements, privacy constraints, and available hardware:

Scenario	Strategy	Rationale
Model ≤3B, privacy required	Pure on-device	Fits in mobile RAM; no network needed; ExecuTorch/llama.cpp
Model 3-8B, flagship device	On-device with context limits	Q4 fits; cap context at 2-4K; monitor thermals
Model 3-8B, browser	WebGPU on-device	Q4_K_M via WebLLM; progressive fallback to WASM → server
Model 8B+, latency tolerant	Distributed (Exo/Petals)	Split across local devices; RDMA for low latency
Model 8B+, low latency needed	Hybrid speculative decoding	Draft on device, verify on cloud; FlexSpec/DSSD
Model 70B+	Cloud with edge caching	Too large for device; cache frequent responses locally
Multi-device home lab	llama.cpp RPC / Exo	Distribute across GPUs over LAN; Thunderbolt for lowest latency
Privacy + quality critical	On-device + cloud fallback	Default on-device; fall back to cloud for hard queries

Framework Comparison Matrix

Framework	Web	Android	iOS	Distributed	Quant Formats	Maturity
ExecuTorch	—	Full	Full	—	.pte (INT4/8, FP16)	GA 1.0 (Oct 2025)
llama.cpp	WASM/Web GPU	NDK	Metal	RPC backend	GGUF (all quants)	Very mature

Framework	Web	Android	iOS	Distributed	Quant Formats	Maturity
MLC-LLM / WebLLM	WebGPU	OpenCL	Metal	—	MLC compiled	Production-ready
MLX	—	—	Native (Metal)	—	MLX format	WWDC 2025, growing
Transformers.js	WebGPU+CPU	—	—	—	ONNX	Mature for web
ORT Web	WebGPU+WebNN	—	—	—	ONNX INT4/8	Production
Exo	—	Planned	Supported	P2P cluster	MLX/tinygrad	Active development
Petals	—	—	—	P2P volunteer	HF formats	Research → production
vLLM	—	—	—	Head/worker	GPTQ/AWQ/FP16	Production serving

Recommended Tech Stack by Use Case

Use Case	Primary Stack	Fallback / Hybrid
Mobile chat app (privacy-first)	ExecuTorch + CoreML (iOS) / XNNPACK (Android); 3B Q4	Cloud API for complex queries
Browser AI assistant	WebLLM (WebGPU) + Phi-3 mini Q4_K_M	WASM fallback → server inference
Self-hosted home cluster	Exo (P2P) or llama.cpp RPC; 70B across multiple GPUs	vLLM for production serving
Enterprise edge deployment	ExecuTorch + Qualcomm QNN; custom fine-tuned 3B	NVIDIA FLARE for federated updates
Real-time AR/translation	On-device 1B Q4; ExecuTorch + ANE (iOS) / NPU (Android)	No cloud (latency constraint)
Research / experimentation	MLX (Apple Silicon) or llama.cpp (cross-platform)	Petals for large model access

11. Key Resources & Links

Inference Frameworks & Runtimes

llama.cpp — CPU/GPU inference, GGUF format, RPC distributed backend

<https://github.com/ggml-org/llama.cpp>

ExecuTorch — PyTorch edge inference, CoreML/XNNPACK/Vulkan/QNN backends

<https://github.com/pytorch/executorch>

MLC-LLM — Cross-platform ML compilation, WebGPU/Metal/OpenCL/Vulkan

<https://github.com/mlc-ai/mlc-llm>

WebLLM — In-browser LLM inference via WebGPU, OpenAI-compatible API

<https://github.com/mlc-ai/web-llm>

MLX — Apple Silicon ML framework, unified memory, Swift API

<https://github.com/ml-explore/mlx>

Transformers.js — HuggingFace models in browser, WebGPU + ONNX

<https://github.com/huggingface/transformers.js>

ONNX Runtime Web — WebGPU/WebNN execution providers, cross-platform
<https://github.com/microsoft/onnxruntime>

vLLM — High-throughput LLM serving, PagedAttention, multi-GPU
<https://github.com/vllm-project/vllm>

PocketPal AI — Open-source mobile LLM chat app (React Native + llama.cpp)
<https://github.com/a-ghorbani/pocketpal-ai>

Distributed Inference

Exo — P2P heterogeneous device clustering, RDMA/Thunderbolt, tensor parallelism
<https://github.com/exo-explore/exo>

Petals — BitTorrent-style distributed LLM, volunteer GPU network
<https://github.com/bigscience-workshop/petals>

EdgeShard — Dynamic programming partitioning for edge-cloud LLM inference
<https://arxiv.org/abs/2405.14371>

MDI-LLM — Recurrent pipeline parallelism for edge LLM inference
<https://arxiv.org/html/2505.18164v1>

Hybrid Edge-Cloud & Speculative Decoding

FlexSpec — Shared-backbone edge-cloud speculative decoding
<https://arxiv.org/abs/2601.00644>

DSSD — Distributed split speculative decoding (device-edge-cloud)
<https://icml.cc/virtual/2025/47046>

EdgeSD — Speculative decoding for multimodal LLMs at edge
<https://ieeexplore.ieee.org/document/11409366/>

NVIDIA FLARE + ExecuTorch — Federated learning on mobile devices
<https://developer.nvidia.com/blog/effortless-federated-learning-on-mobile-with-nvidia-flare-and-meta-executorch/>

Reference Articles & Benchmarks

On-Device LLMs: State of the Union, 2026 — Comprehensive survey of on-device LLM landscape
<https://v-chandra.github.io/on-device-llms/>

Ship Private, Offline AI in 2025 (debugg.ai) — Production blueprint for browser-based LLM inference
<https://debugg.ai/resources/ship-private-offline-ai-2025-on-device-llm-webgpu-webnn-wasm>

llama.cpp Apple Silicon Benchmarks — Community performance benchmarks across Apple devices
<https://github.com/ggml-org/llama.cpp/discussions/4508>

WebNN W3C Specification — Web Neural Network API specification
<https://www.w3.org/TR/webnn/>

WebNN Overview (Microsoft) — WebNN integration with DirectML on Windows
<https://learn.microsoft.com/en-us/windows/ai/directml/webnn-overview>

Quantization & Optimization

GGUF / llama.cpp quantize — GGUF quantization tool documentation
<https://github.com/ggml-org/llama.cpp/blob/master/tools/quantize/README.md>

AWQ (Activation-aware Weight Quantization) — 4-bit quantization preserving salient weights
<https://github.com/mit-han-lab/llm-awq>

GPTQ — Post-training quantization for GPT models

<https://github.com/IST-DASLab/gptq>

BitNet — Native 1.58-bit training and inference

<https://github.com/microsoft/BitNet>